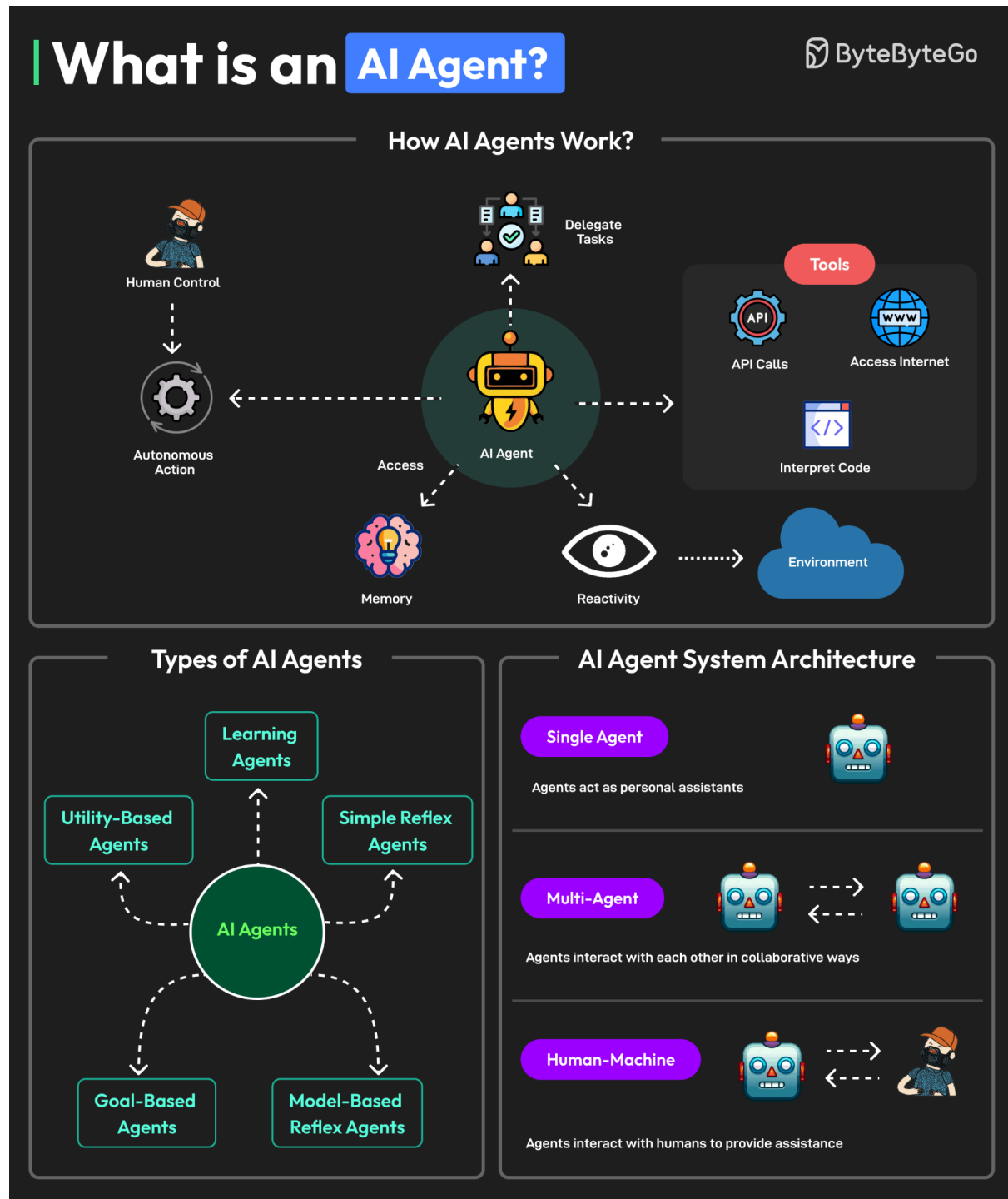


What is an AI Agent?



An AI agent is a software program that can interact with its environment, gather data, and use that data to achieve predetermined goals. AI agents can choose the best actions to perform to meet those goals.

Key characteristics of AI agents are as follows:

An agent can perform autonomous actions without constant human intervention. Also, they can have a human in the loop to maintain control.

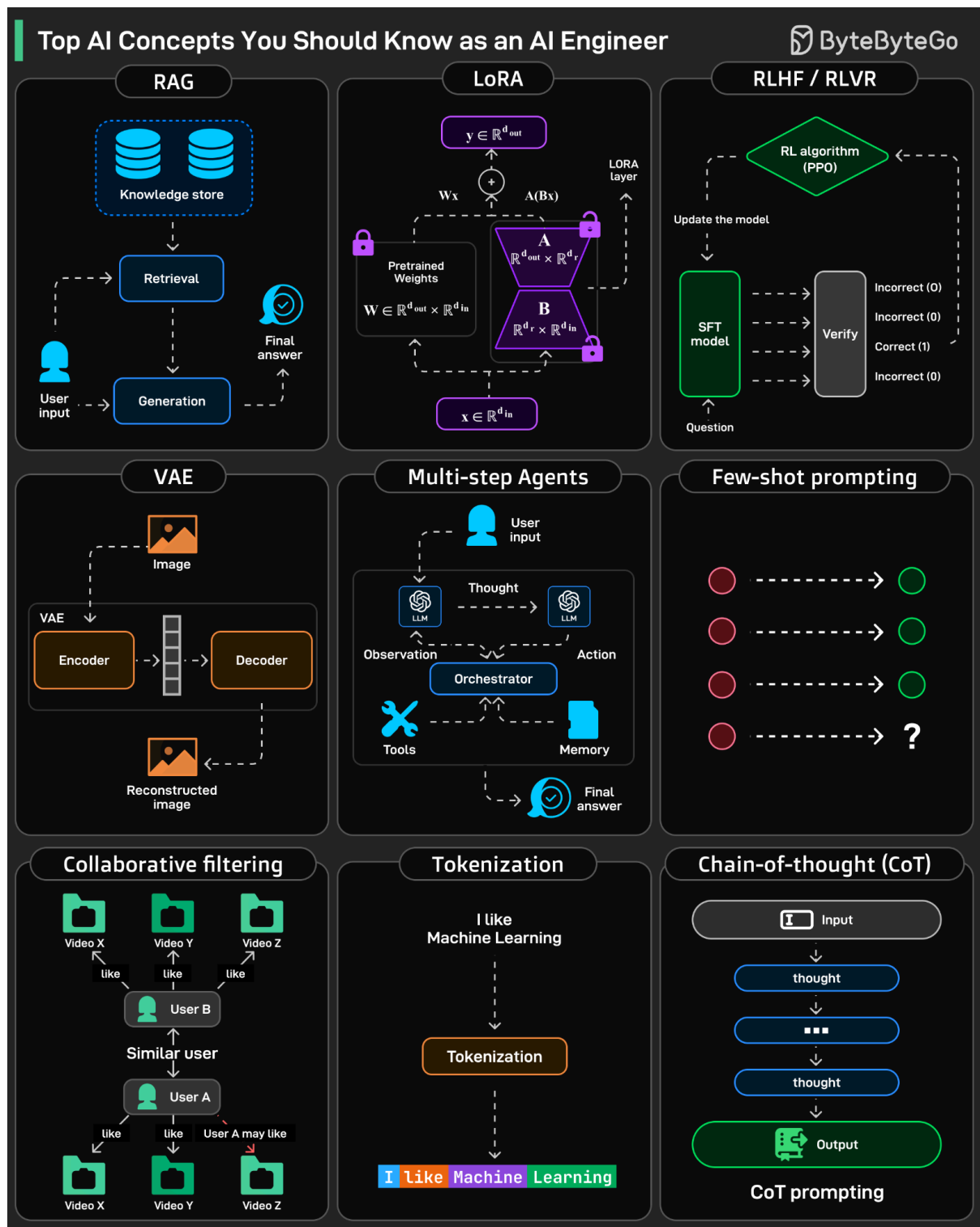
- Agents have a memory to store individual preferences and allow for personalization. It can also store knowledge. An LLM can undertake information processing and decision-making functions.
- Agents must be able to perceive and process the information available from their environment.
- Agents can also use tools such as accessing the internet, using code interpreters and making API calls.
- Agents can also collaborate with other agents or humans.

Multiple types of AI agents are available such as learning agents, simple reflex agents, model-based reflex agents, goal-based agents, and utility-based agents.

A system with AI agents can be built with different architectural approaches.

1. **Single Agent:** Agents can serve as personal assistants.
2. **Multi-Agent:** Agents can interact with each other in collaborative or competitive ways.
3. **Human Machine:** Agents can interact with humans to execute tasks more efficiently.

9 Key AI Concepts Explained



What are the core concepts that keep showing up in AI products?

Many AI systems reuse the same building blocks, just wired together differently. This visual is a map of nine of the most common ones.

CoT and few-shot prompting improve LLMs' outputs by giving the model a pattern to follow.

RAG and agents connect a static LLM to knowledge, tools, and actions in the real world to make them reliable and powerful.

RLHF and RLVR make LLMs useful by letting them practice, then rewarding what humans or verifiers prefer.

Tokenization is the hidden interface that decides what the model sees.

Collaborative filtering recommends the next best item from clicks and co-occurrence

Most AI engineering work is choosing which blocks to combine, and where.

Top 20 AI Agent Concepts You Should Know

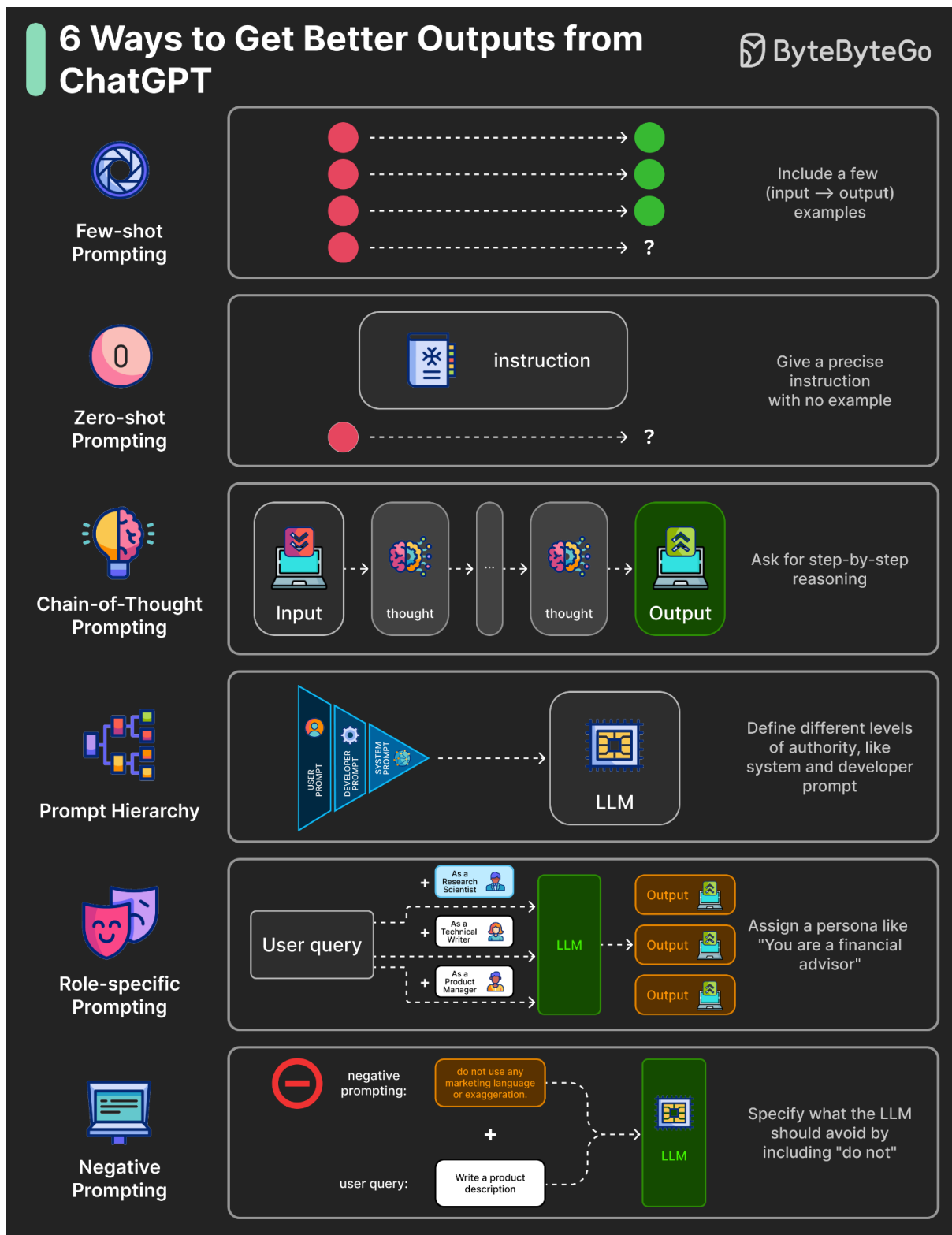
Top 20 AI Agent Concepts You Should Know 			
Agent 1  Autonomous entity that perceives, reasons and acts to achieve goals	Environment 2  Surrounding context in which the agent operates and interacts	Perception 3  Process of an agent interpreting sensory or environmental data	State 4  Agent's current internal condition or representation of the world
Memory 5  Storage of recent or historical information for continuity and learning	Large Language Models 6  Foundation models powering language understanding and generation	Reflex Agent 7  An agent that makes decisions based on predefined "condition-action" rules	Knowledge Base 8  Structured or unstructured data repository used by agents to make decisions
CoT (Chain of Thought) 9  Reasoning method where agents articulate intermediate steps for complex tasks	ReAct 10  A framework that combines step-by-step reasoning with environmental actions	Tools 11  APIs or external systems that agents use to augment capabilities	Action 12  Any task or behavior executed by the agent
Planning 13  Devising a sequence of actions to reach a specific goal	Orchestration 14  Coordinating multiple steps, tools, or agents to fulfill a task pipeline	Handoffs 15  The transfer of responsibilities or tasks between different agents	Multi-Agent System 16  A framework where multiple agents operate and collaborate in the same environment
Swarm 17  Emergent intelligent behavior from many agents following local rules without central control	Agent Debate 18  Mechanism where agents argue opposing view to refine or improve final response	Evaluation 19  Measuring the effectiveness of an agent's actions	Learning Loop 20  The cycle where agents improve performance by continuously learning from feedback

1 - Agent: An autonomous entity that perceives, reasons, and acts in an environment to achieve goals.

2 - Environment: The surrounding context or sandbox in which the agent operates and interacts.

- 3 - Perception:** The process of interpreting sensory or environmental data to build situational awareness.
- 4 - State:** The agent's current internal condition or representation of the world.
- 5 - Memory:** Storage of recent or historical information for continuity and learning.
- 6 - Large Language Models:** Foundation models powering language understanding and generation.
- 7 - Reflex Agent:** A simple type of agent that makes decisions based on predefined "condition-action" rules.
- 8 - Knowledge Base:** Structured or unstructured data repository used by agents to inform decisions.
- 9 - CoT (Chain of Thought):** A reasoning method where agents articulate intermediate steps for complex tasks.
- 10 - ReAct:** A framework that combines step-by-step reasoning with direct environmental actions.
- 11 - Tools:** APIs or external systems that agents use to augment their capabilities.
- 12 - Action:** Any task or behavior executed by the agent as a result of its reasoning.
- 13 - Planning:** Devising a sequence of actions to reach a specific goal.
- 14 - Orchestration:** Coordinating multiple steps, tools, or agents to fulfill a task pipeline.
- 15 - Handoffs:** The transfer of responsibilities or tasks between different agents.
- 16 - Multi-Agent System:** A framework where multiple agents operate and collaborate in the same environment.
- 17 - Swarm:** Emergent intelligent behavior from many agents following local rules without central control.
- 18 - Agent Debate:** A mechanism where agents argue opposing views to refine or improve outcomes.
- 19 - Evaluation:** Measuring the effectiveness or success of an agent's actions and outcomes.
- 20 - Learning Loop:** The cycle where agents improve performance by continuously learning from feedback or outcomes.

Key Prompt Engineering Techniques



LLMs are powerful, but their answers depend on how the question is asked. Prompt engineering adds clear instructions that set goals, rules, and style. This turns vague questions and tasks into clear, well-defined prompts.

What are the key prompt engineering techniques?

1. Few-shot Prompting: Include a few (input → output) example pairs in the prompt to teach the pattern.

2. Zero-shot Prompting: Give a precise instruction without examples to state the task clearly.

3. Chain-of-thought (CoT) Prompting: Ask for step-by-step reasoning before the final answer. This can be zero-shot, where we explicitly include “Think step by step” in the instruction, or few-shot, where we show some examples with step-by-step reasoning.

4. Role-specific Prompting: Assign a persona, like “You are a financial advisor,” to set context for the LLM.

5. Prompt Hierarchy: Define system, developer, and user instructions with different levels of authority. System prompts define high-level goals and set guardrails, while developer prompts define formatting rules and customize the LLM’s behavior.

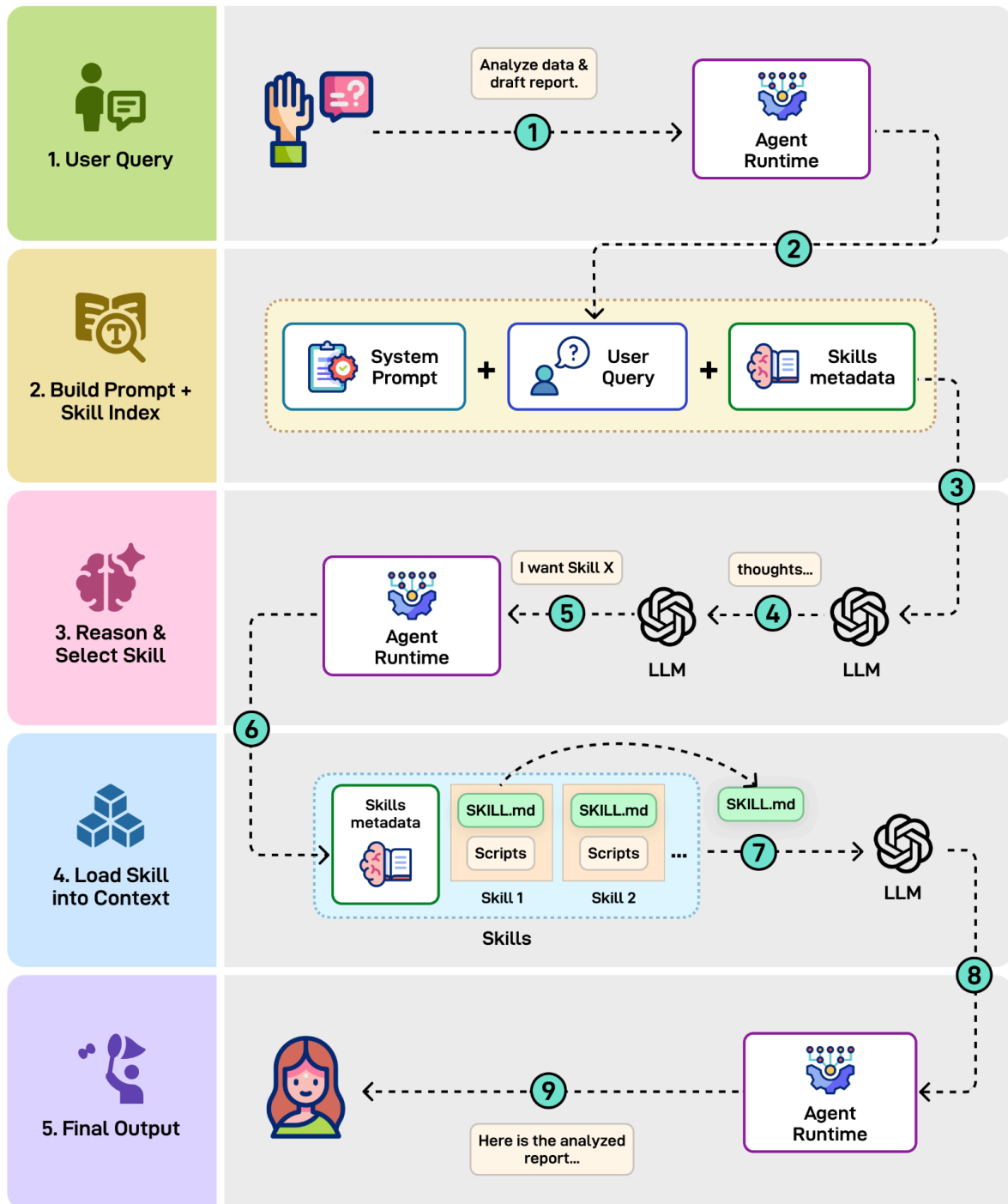
Here are the key principles to keep in mind when engineering your prompts:

- Begin simple, then refine.
- Break a big task into smaller, more manageable subtasks.
- Be specific about desired format, tone, and success criteria.
- Provide just enough context to remove ambiguity.

Agent Skills, Clearly Explained

What are Agent Skills?

ByteByteGo



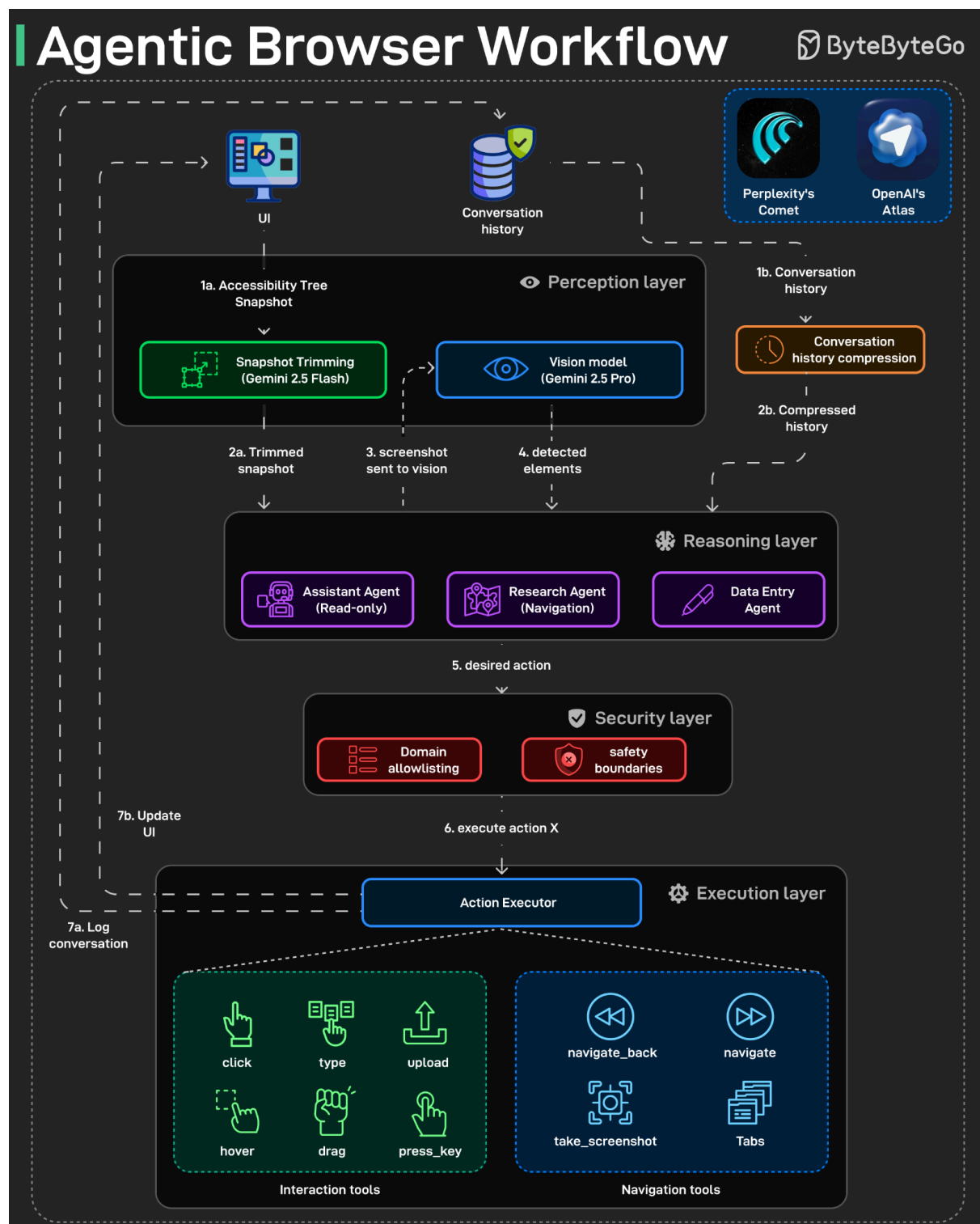
Why do we need Agent Skills? Long prompts hurt agent performance. Instead of one massive prompt, agents keep a small catalog of skills, reusable playbooks with clear instructions, loaded only when needed.

Here is what the Agent Skills workflow looks like:

- 1. User Query:** A user submits a request like “Analyze data & draft report”.
- 2. Build Prompt + Skills Index:** The agent runtime combines the query with Skills metadata, a lightweight list of available skills and their short descriptions.
- 3. Reason & Select Skill:** The LLM processes the prompt, thinks, and decides: "I want Skill X."
- 4. Load Skill into Context:** The agent runtime receives the specific skill request from the LLM. Then, it loads SKILL.md and adds it into the LLM's active context.
- 5. Final Output:** The LLM follows SKILL.md, runs scripts, and generates the final report.

By dynamically loading skills only when needed, Agent Skills keep context small and the LLM's behavior consistent.

How Agentic Browsers like OpenAI's Atlas Work



Agentic browsers like OpenAI Atlas and Perplexity Comet embed an agent that can read webpages and take actions in your browser.

Most agentic browsers have four major layers.

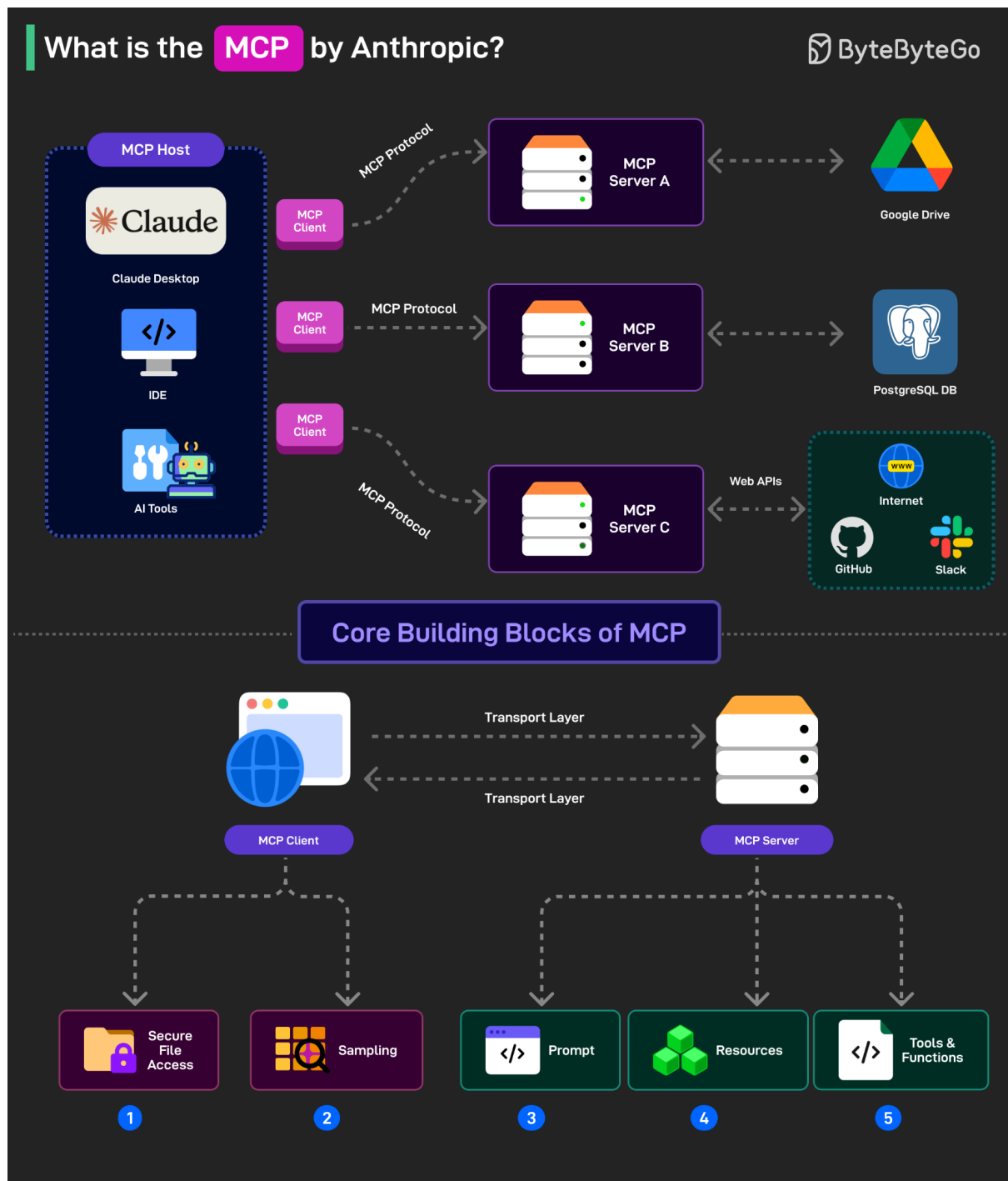
1. Perception layer: Converts the current UI into model input. It starts with an accessibility tree snapshot. If the tree is incomplete or ambiguous, the agent takes a screenshot, sends it to a vision model (for example, Gemini Pro) to extract UI elements into a structured form, then uses that result to decide the next action.

2. Reasoning layer: Uses specialized agents for read-only browsing, navigation, and data entry. Separating roles improves reliability and lets you apply safety rules per agent.

3. Security layer: Enforces domain allowlisting and deterministic boundaries such as restricted actions, and confirmation steps to reduce prompt injection risk.

4. Execution layer: Runs browser tools (click, type, upload, navigate, screenshot, tab operations) and refreshes state after each step.

What is MCP?



Model Context Protocol (MCP) is a new system introduced by Anthropic to make AI models more powerful.

It is an open standard (also being run as an open-source project) that allows AI models (like Claude) to connect to databases, APIs, file systems, and other tools without needing custom code for each new integration.

MCP follows a client-server model with 3 key components:

1 - Host: AI applications like Claude that provide the environment for AI interactions so that different tools and data sources can be accessed. The host runs the MCP Client.

2 - MCP Client: The MCP client is the component inside an AI model (like Claude) that allows it to communicate with MCP servers. For example, if the AI model wants data from PostgreSQL, the MCP client formats the request into a structured message to send to the MCP Server

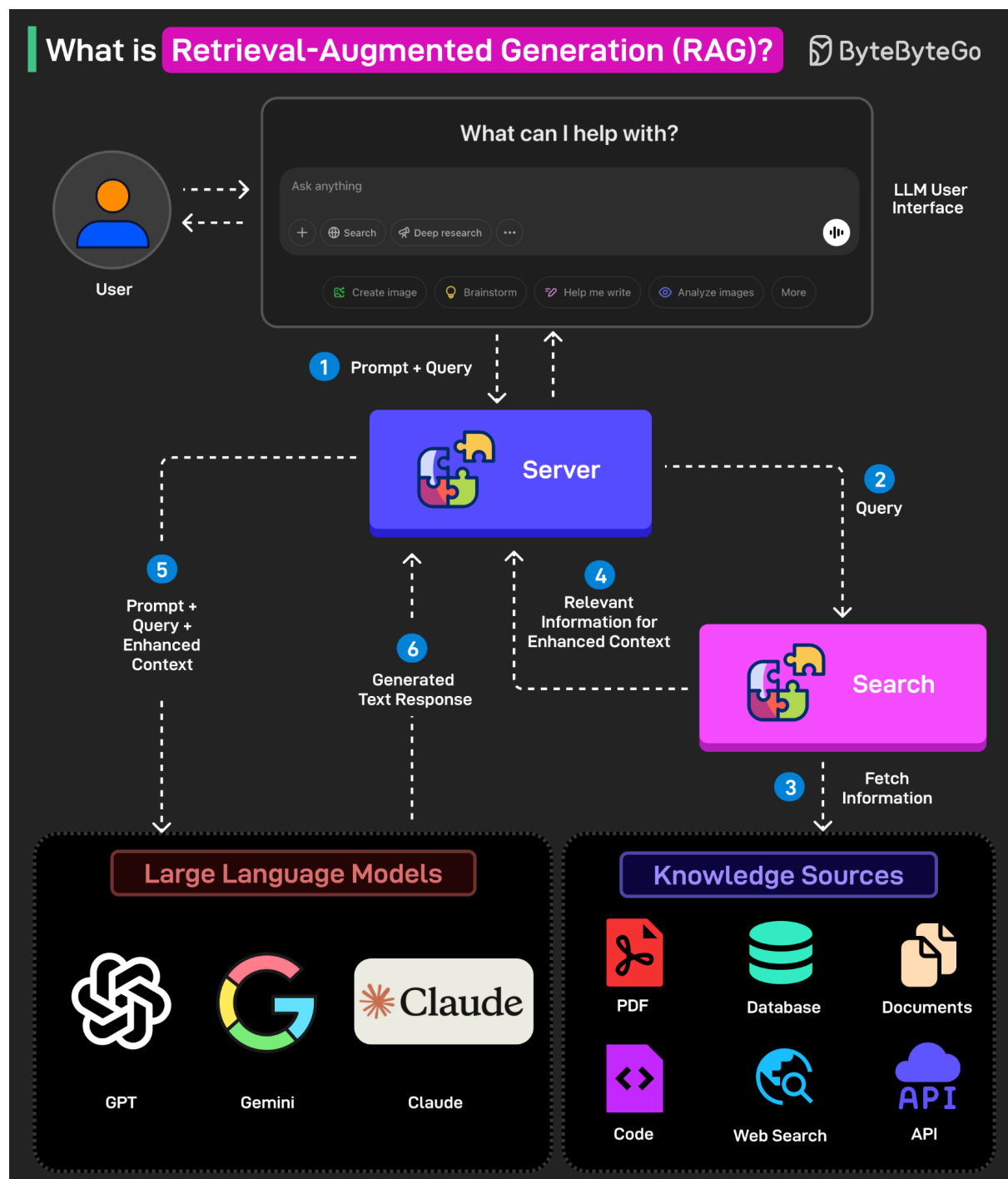
3 - MCP Server: This is the middleman that connects an AI model to an external system like PostgreSQL, Google Drive, or an API. For example, if Claude analyzes sales data from PostgreSQL, the MCP Server for PostgreSQL acts as the connector between Claude and the database.

MCP has five core building blocks (also known as primitives). They are divided between the client and server.

1 - For the clients, the building blocks are Roots (secure file access) and Sampling (ask the AI for help with a task such as generating a DB query).

2 - For the servers, there are Prompts (instructions to guide the AI), Resources (Data Objects that the AI can reference) and Tools (functions that the AI can call such as running a DB query).

What is Retrieval-Augmented Generation (RAG)?



RAG is the process of optimizing an LLM's output to reference a specific knowledge base that may not have been part of its training data before generating a response.

In other words, RAG helps extend the powerful capabilities of LLMs to specific domains or knowledge bases without the need for additional training.

Here's how RAG works:

1 - The user writes a query prompt in the LLM's user interface. This query is passed to the backend server where it is converted to a vector representation.

2 - The query is sent to a search system.

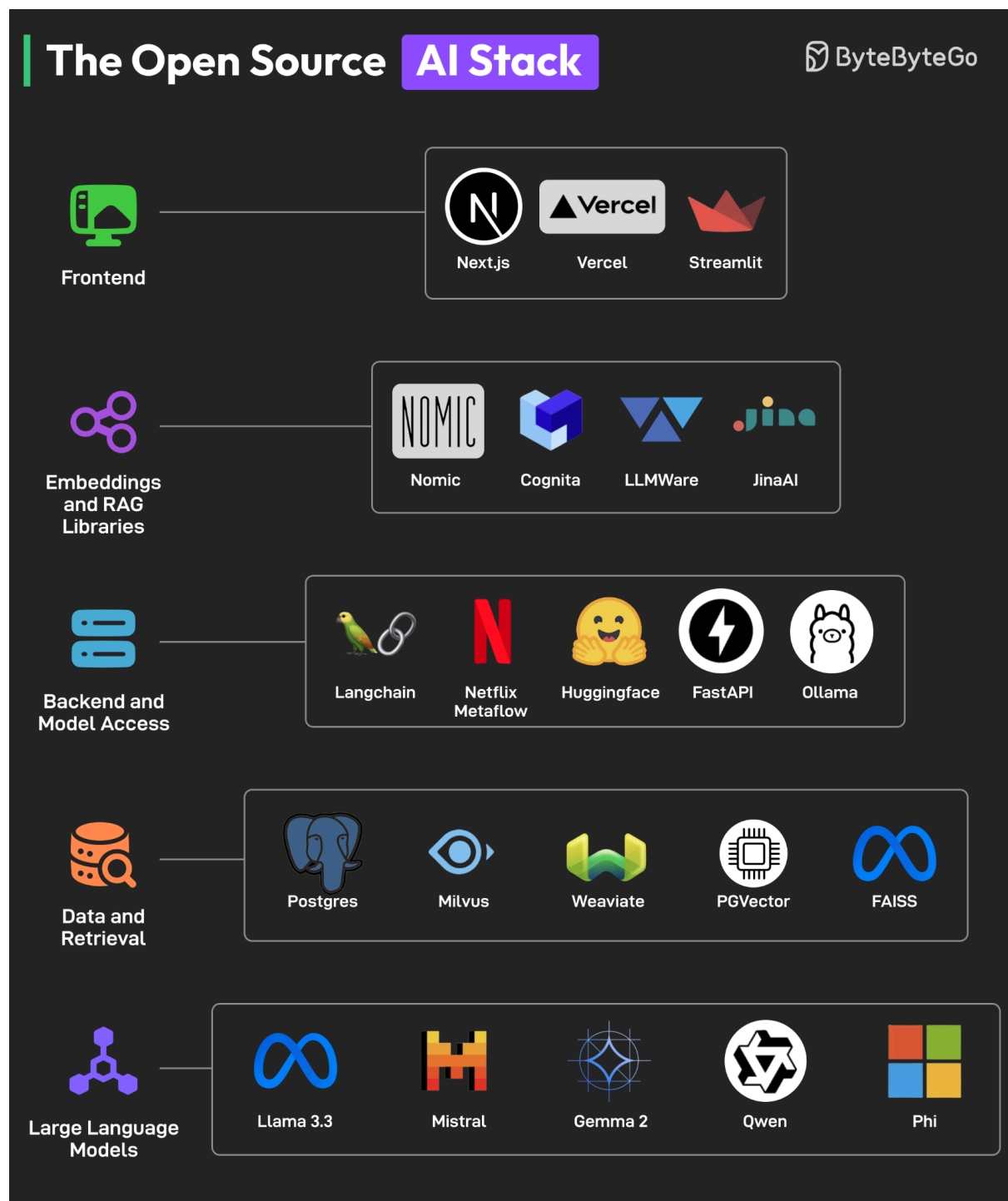
3 - This search system can refer to various knowledge sources such as PDFs, Web Search, Code-bases, Documents, Database, or APIs to fetch relevant information to answer the query.

4 - The fetched information is sent back to the RAG model.

5 - The model augments the original user input by adding the fetched information to the context and sending it to the LLM endpoint. Various LLM options are Open AI's GPT, Claude Sonnet, Google Gemini, and so on.

6 - The LLM generates an answer based on the enhanced context and provides a response to the user.

The Open-Source AI Stack



You don't need to spend a fortune to build an AI application. The best AI developer tools are open-source, and an excellent ecosystem is evolving that can make AI accessible to everyone.

The key components of this open-source AI stack are as follows:

1 - Frontend

To build beautiful AI UIs, frameworks like NextJS and Streamlit are extremely useful. Also, Vercel can help with deployment.

2 - Embeddings and RAG libraries

Embedding models and RAG libraries like Nomic, JinaAI, Cognito, and LLMAware help developers build accurate search and RAG features.

3 - Backend and Model Access

For backend development, developers can rely on frameworks like FastAPI, Langchain, and Netflix Metaflow. Options like Ollama and Huggingface are available for model access.

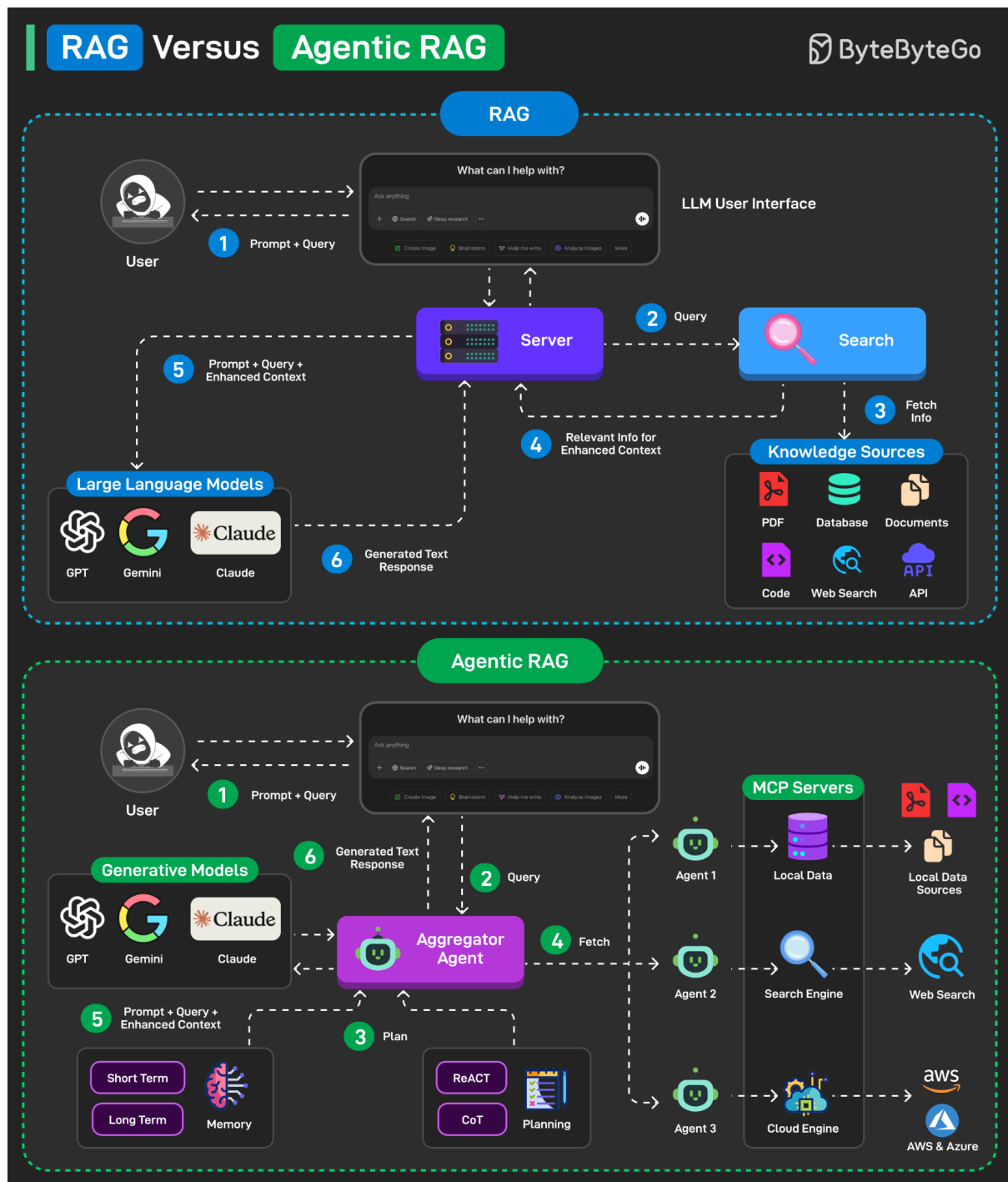
4 - Data and Retrieval

For data storage and retrieval, several options like Postgres, Milvus, Weaviate, PGVector, and FAISS are available.

5 - Large-Language Models

Based on performance benchmarks, open-source models like Llama, Mistral, Qwen, Phi, and Gemma are great alternatives to proprietary LLMs like GPT and Claude.

RAG vs Agentic RAG



RAG (Retrieval Augmented Generation) is a method that combines information retrieval with large language models to generate answers. Here's how RAG works on a high level:

1 - The model retrieves relevant data from data sources and then extracts it to a vector database from the pre-indexed model.

2 - Augment the prompts by retrieving information and merging it with the query prompt.

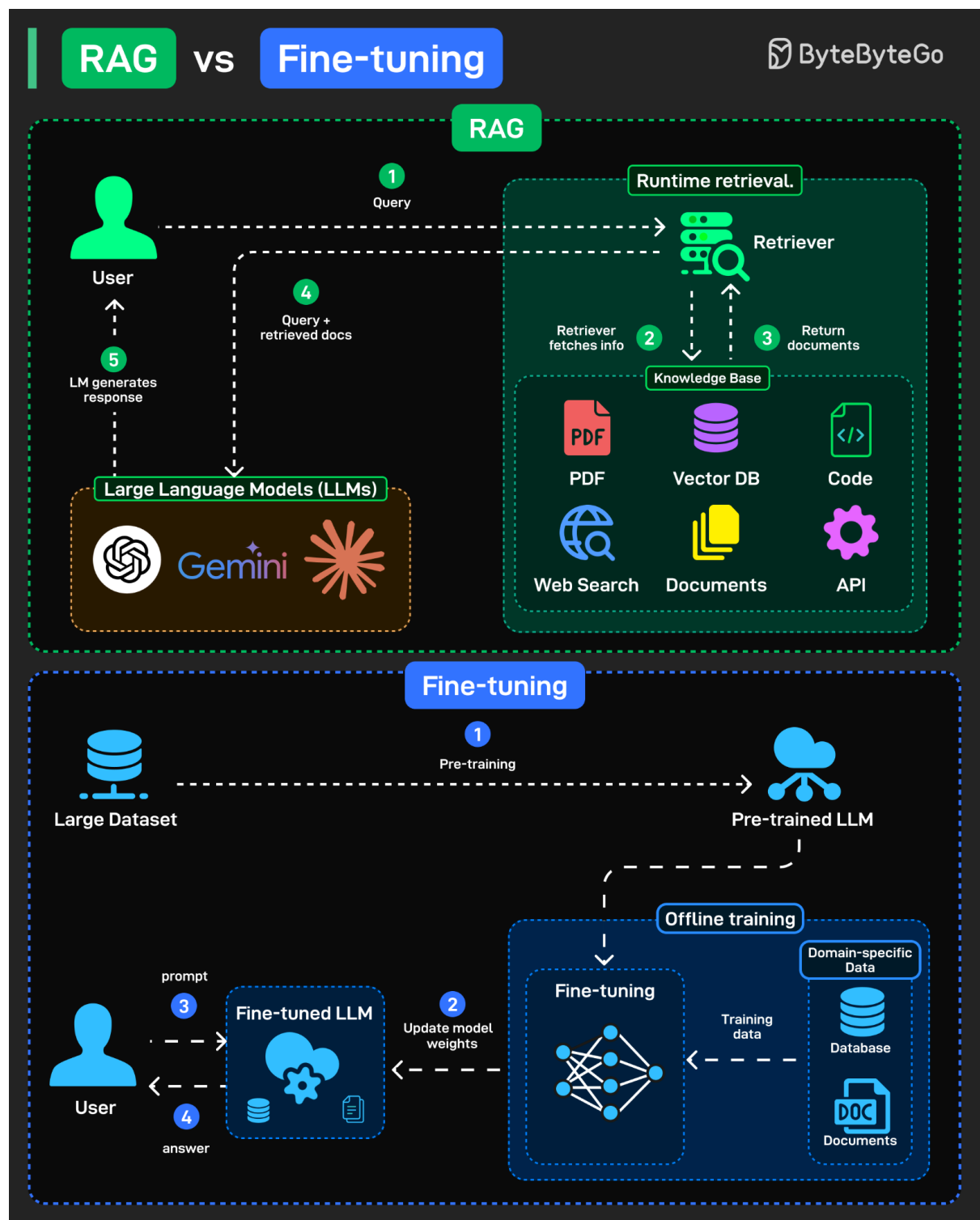
3 - A Large Language Model (like GPT, Claude, or Gemini) understands the combined query and generates the final response.

A traditional RAG has a simple retrieval, limited adaptability, and relies on static knowledge, making it less flexible for dynamic and real-time information.

Agentic RAG improves on this by introducing AI agents that can make decisions, select tools, and even refine queries for more accurate and flexible responses. Here's how Agentic RAG works on a high level:

1. The user query is directed to an AI Agent for processing.
2. The agent uses short-term and long-term memory to track query context. It also formulates a retrieval strategy and selects appropriate tools for the job.
3. The data fetching process can use tools such as vector search, multiple agents, and MCP servers to gather relevant data from the knowledge base.
4. The agent then combines retrieved data with a query and system prompt. It passes this data to the LLM.
5. LLM processes the optimized input to answer the user's query.

RAG vs Fine-tuning



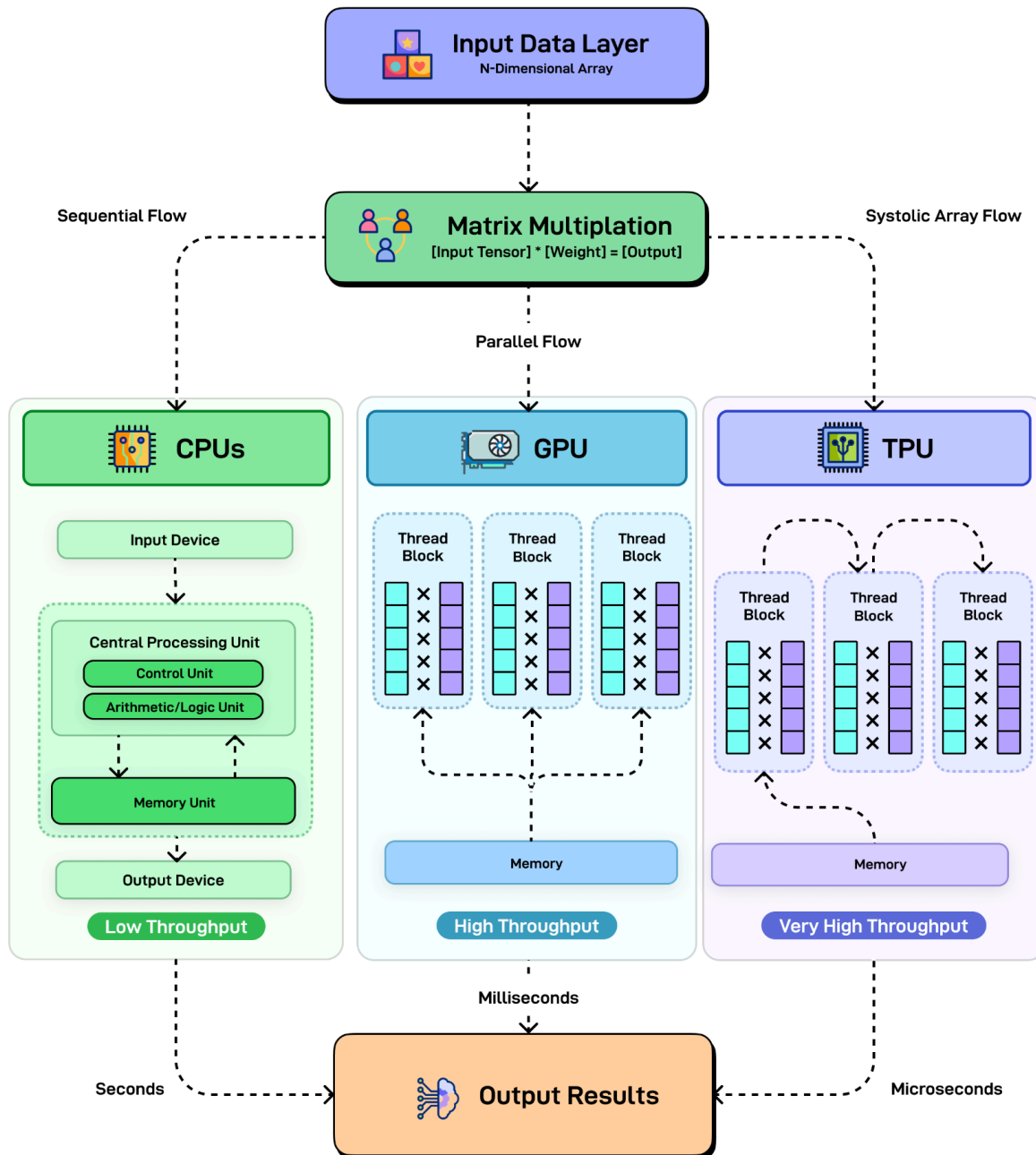
Which one should you use?

When it comes to adapting Large Language Models (LLMs) to new tasks, two popular approaches stand out: Retrieval-Augmented Generation (RAG) and Fine-tuning. They solve the same problem, making models more useful, but in very different ways.

- **RAG (Retrieval-Augmented Generation):** Fetches knowledge at runtime from external sources (docs, DBs, APIs). Flexible, always fresh.
- **Fine-tuning:** Offline training that updates model weights with domain-specific data, making the model an expert in your field.

Why Does AI Need GPUs and TPUs?

Why AI Needs GPUs and TPUs



When AI processes data, it's essentially multiplying massive arrays of numbers. This can mean billions of calculations happening simultaneously.

CPUs can handle such calculations sequentially. To perform any calculation, the CPU must fetch an instruction, retrieve data from memory, execute the operation, and write results back. This constant transfer of information between the processor and memory is not very efficient. It's like having one very smart person solve a giant puzzle alone.

GPUs change the game with parallel processing. They split the work across hundreds of cores, reducing processing time to milliseconds.

However, TPUs take it even further with a systolic array architecture that's thousands of times faster than CPUs. Each unit in a TPU multiplies its stored weight by incoming data, adds to a running sum flowing vertically, and passes both values to its neighbor. This cuts down on I/O costs and reduces processing times even further.